DIGITAL EVIDENCE MANAGEMENT SYSTEM

21CSC205P Database Management Systems
A MINI PROJECT REPORT

SAI KRISHNA [RA2411026020505]
RUDRASH RAJPUROHIT [RA2411026010506]

Dr.A. Sheryl Oliver
Assistant Professor, CINTEL

in partial fulfillment of the requirementsfor the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Artificial Intelligence and Machine Learning

DEPARTMENT OF COMPUTATIONAL INTELLIGENCE
SCHOOL OF COMPUTING
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE ANDTECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGYKATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that Project report titled "**DIGITAL EVIDENCE MANAGEMENT SYSTEM**" is the bonafide work of "**SAI KRISHNA [RA2411026010505],RUDRASH RAJPUROHIT[RA2411026010506]**" who carried out the **21CSC205P Database Management Systems** mini project work under my supervision.

COURSE FACULTY
DESIGNATION
Department

HEAD OF THE DEPARMENT
PROFESSOR AND HEAD
Department

ABSTRACT

The Digital Evidence Management System (DEMS) is a secure and reliable database-driven platform designed to store, manage, and monitor digital evidence efficiently. Digital evidence, such as documents, images, videos, emails, and system logs, plays a crucial role in forensic investigations, legal proceedings, and organizational audits. However, traditional evidence management systems often face challenges such as unauthorized access, data tampering, lack of proper tracking, and inefficient access control. The DEMS addresses these issues by providing a structured and secure environment for handling sensitive digital evidence.

The system uses a centralized database to store evidence metadata, user details, access logs, permissions, and chain of custody records. Each user is assigned a specific role, and access to evidence is controlled through role-based access control mechanisms. This ensures that only authorized users can upload, view, download, or manage evidence based on their permissions. Every action performed on evidence is recorded in the access log, including the user identity, access type, timestamp, and IP address, ensuring complete accountability and transparency.

The system also maintains a chain of custody, which tracks the complete history of evidence handling, including transfers, reviews, and modifications. This is essential for maintaining the legal integrity and authenticity of evidence. Additionally, watermarking functionality is implemented to protect sensitive evidence from unauthorized sharing by embedding identifiable information into the files. The notification module alerts users and administrators about important events such as new uploads, access attempts, or system updates.

The Digital Evidence Management System is designed using DBMS concepts such as entity-relationship modeling, normalization, access control, and audit logging. The system ensures data integrity, confidentiality, and availability while providing efficient and secure evidence management. This project demonstrates the practical application of database management systems in real-world scenarios, particularly in forensic and investigative environments, where secure handling and tracking of digital evidence are critical.

TABLE OF CONTENTS

Chapter No.	Title	Page no.
	ABSTRACT	
1	Problem understanding, Identification of Entity and Relationships, Construction of DB using ER Model	
2	Design of Relational Schemas, Creation of Database and Tables for their project	
3	3.1Writing queries based on the concepts of Aggregate functions, constraints, sets	
	3.2 Writing complex queries based on subqueries, joins and views	
	3.3 Writing complex queries based on the concepts of functions, Triggers, Cursors and Exception Handling	
4	4.1 Analysing the pitfalls, identifying the dependencies, and applying Normalizations- 1NF, 2NF, 3NF	
	4.2 Analysing the pitfalls, identifying the dependencies, and applying Normalizations- 4NF, 5NF	
5	Implementation of concurrency control and recovery mechanisms.	
6	Front-end And Back-End Implementation code	
7	Results and Discussions	

LIST OF FIGURES

Figure No.	Name	Page no.
1		
2		
3		
4		
5		

LIST OF TABLES

Figure No.	Name	Page no.
1		
2		
3		
4		
5		

CHAPTER 1

PROBLEM UNDERSTANDING, IDENTIFICATION OF ENTITY AND RELATIONSHIPS, CONSTRUCTION OF DB USING ER MODEL FOR THE DIGITAL EVIDENCE MANAGEMENT SYSTEM

1.1 Introduction

The Digital Evidence Management System (DEMS) is a secure and database-driven system designed to store, manage, and monitor digital evidence in an organized and controlled manner. Digital evidence refers to any information stored in digital form that can be used in investigations, audits, or legal proceedings, such as documents, images, videos, emails, and system logs. Managing such evidence requires high levels of security, integrity, and accountability to ensure that the evidence is not tampered with or accessed by unauthorized users.

The DEMS provides a centralized platform where users can upload, access, and manage digital evidence based on their authorized roles. The system uses database management principles to maintain structured storage of evidence, user information, access logs, permissions, and chain of custody records. Each interaction with evidence is recorded, ensuring complete traceability and accountability.

The system also includes features such as role-based access control, audit logging, watermarking, and notification management to enhance security and monitoring. By using a structured database approach, the system ensures data integrity, confidentiality, and

availability. This project demonstrates the practical implementation of database management concepts in real-world forensic and investigative environments.

1.2 Motivation

The increasing use of digital devices has led to a significant rise in digital evidence in investigations and organizational processes. Traditional methods of storing and managing evidence are often insecure, unorganized, and vulnerable to unauthorized access or data loss. There is a need for a secure and efficient system that can manage digital evidence while maintaining its integrity and authenticity.

The motivation behind developing the Digital Evidence Management System is to provide a reliable platform that ensures secure storage, controlled access, and proper tracking of digital evidence. The system helps prevent unauthorized access, ensures accountability, and maintains a proper chain of custody. It also demonstrates the practical application of database management systems in solving real-world security and data management problems.

1.3 Scope

The scope of the Digital Evidence Management System includes secure storage, management, and monitoring of digital evidence using a structured database system. The system allows authorized users to upload, access, and manage evidence based on their assigned roles and permissions.

The system maintains access logs to track all activities related to evidence, including viewing, downloading, and uploading. It also maintains chain of custody records to ensure the legal integrity of evidence. The system provides watermarking functionality to protect sensitive evidence and notification features to inform users about important events.

The project focuses on implementing database concepts such as ER modelling, normalization, relationships, and access control. It is suitable for forensic departments, law enforcement agencies, and organizations that require secure evidence management.

1.4 Problem Statement

Managing digital evidence securely and efficiently is a major challenge in forensic and investigative environments. Traditional systems often lack proper access control, audit tracking, and secure storage mechanisms. This can lead to unauthorized access, data tampering, and loss of evidence integrity.

There is a need for a system that can securely store digital evidence, control access based on user roles, maintain complete access logs, and ensure a proper chain of custody. The system should also provide features such as watermarking and notifications to enhance security and monitoring.

The Digital Evidence Management System is designed to solve these problems by providing a secure, structured, and database-driven solution for managing digital evidence.

1.5 Project Requirements

The project requires both functional and non-functional requirements to ensure proper system operation.

Functional Requirements:

- User registration and authentication
- Upload and storage of digital evidence
- Role-based access control
- Evidence access tracking and logging
- Chain of custody management
- Watermark application to evidence
- Notification system for users

Non-Functional Requirements:

• Security and confidentiality
• Data integrity
• Scalability
• Efficient database performance

Hardware Requirements:

• Computer system with minimum 4GB RAM
• Storage device

Software Requirements:

• Database: MySQL
• Frontend: HTML, CSS, JavaScript
• Backend: Python / Node.js
• Operating System: Windows / Linux

1.6 Identification of Entity and Relationships

The main entities identified in the Digital Evidence Management System are:

• User
• Evidence
• Access_Log
• Chain_of_Custody
• Watermark
• Permission
• Notification

Each entity contains specific attributes. For example, the User entity contains user_id, name, email, and role. The Evidence entity contains evidence_id, title, file_path, and confidentiality level.The relationships between entities include:

• User uploads Evidence
• User accesses Evidence (recorded in Access_Log)
• Evidence has Chain_of_Custody records
• User applies Watermark to Evidence
• User has Permission for Evidence
• User receives Notifications

These relationships help maintain structured and secure management of digital evidence.

1.7 Construction of DB Using ER Model for the Digital Evidence Management System

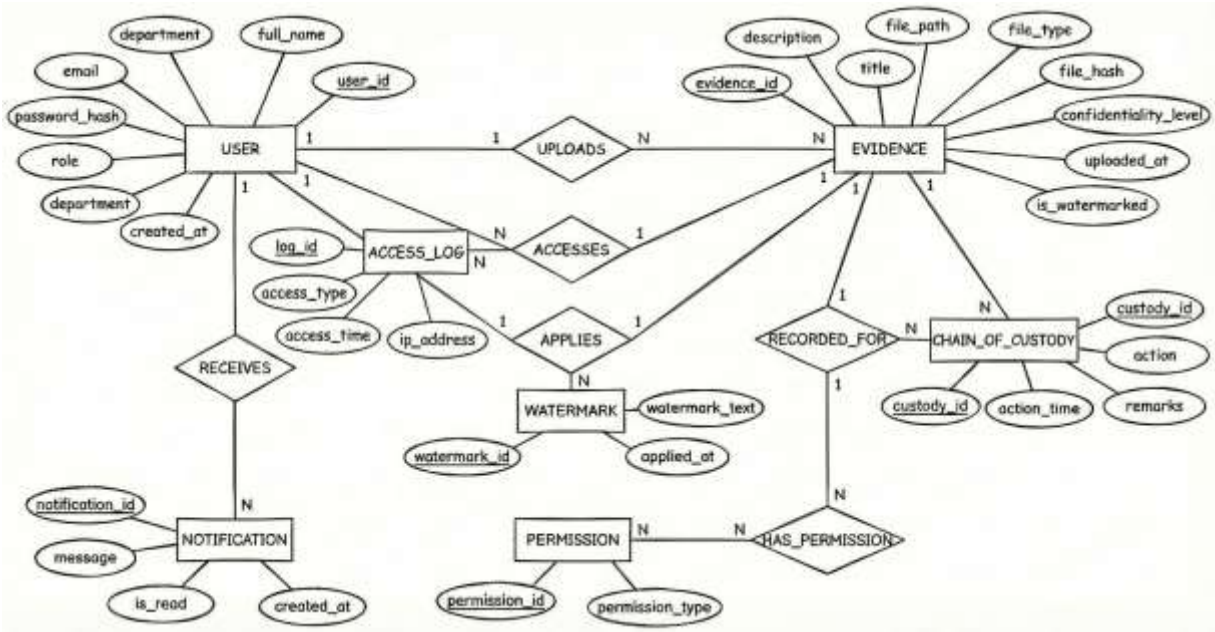
The database for the Digital Evidence Management System is constructed using the Entity-Relationship (ER) model. The ER model represents entities, attributes, and relationships in a graphical form, which helps in designing the database structure clearly.

Each entity is converted into a database table, and attributes become columns in the table. Primary keys are used to uniquely identify each record, and foreign keys are used to establish relationships between tables.

For example:

• User table stores user information
• Evidence table stores evidence details
• Access_Log table stores access history
• Permission table manages user permissions
• Chain_of_Custody table tracks evidence handling
• Notification table stores system alerts

The ER model ensures proper database design, eliminates redundancy, and maintains data integrity. It provides a clear structure for implementing the Digital Evidence Management System using relational database concepts.


department
full_name
email
user_id
password_hash
USER
role
department
created_at
UPLOADS
description
file_path
file_type
evidence_id
title
file_hash
confidentiality_level
EVIDENCE
uploaded_at
is_watermarked
log_id
ACCESS_LOG
ACCESSES
access_type
access_time
ip_address
APPLIES
RECEIVES
RECORDED_FOR
CHAIN_OF_CUSTODY
custody_id
action
custody_id
action_time
remarks
WATERMARK
watermark_text
watermark_id
applied_at
notification_id
NOTIFICATION
message
is_read
created_at
PERMISSION
HAS_PERMISSION
permission_id
permission_type

Chapter 2

Design of Relational Schemas, Creation of Database and Tables for Digital Evidence Management System

2.1 Relational Schema for Digital Evidence Management System

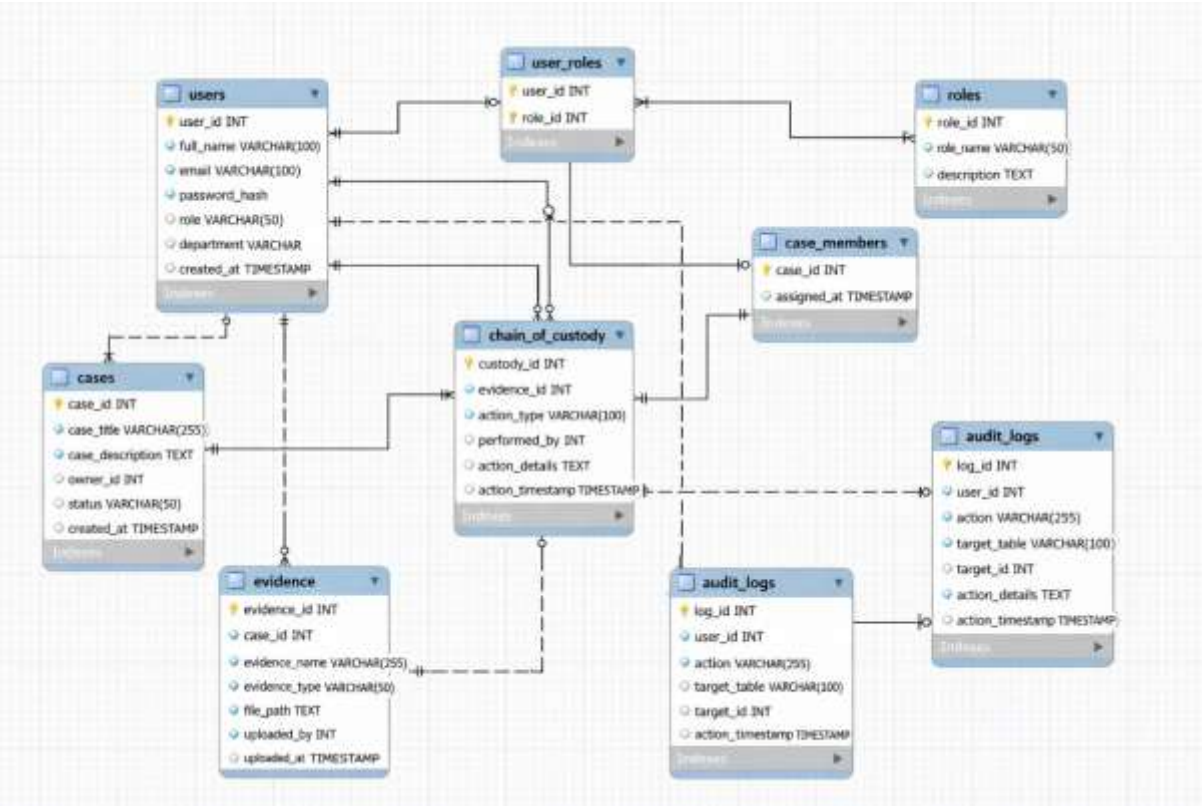


Figure 2.1: Relational Schema for Digital Evidence Management System

The figure 2.1 depicts the relational schema of the **Digital Evidence Management System**. The schema consists of multiple interrelated tables such as **users, roles, user_roles, cases, evidence, chain_of_custody, case_members, and audit_logs**.

Each table represents a specific entity in the system:

- The **users** table stores user information
- The **roles** table defines user roles
- The **user_roles** table maps users to roles (many-to-many relationship)
- The **cases** table manages investigation cases

- The **evidence** table stores digital evidence related to cases
- The **chain_of_custody** table tracks evidence handling history
- The **case_members** table assigns users to cases
- The **audit_logs** table records system activities

The relationships ensure:

- Data integrity
- Proper tracking of evidence
- Secure access control

2.2 Description of Tables

Table 2.1: **Users**

Purpose: Stores details of system users

Primary Key: `user_id`

Attribute	Data Type	Constraints
user_id	INT	PK, NOT NULL, AUTO_INCREMENT
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
role	VARCHAR(50)	
department	VARCHAR(100)	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table 2.2: **Roles**

Purpose: Stores different user roles

Attribute	Data Type	Constraints
role_id	INT	PK
role_name	VARCHAR(50)	NOT NULL
description	TEXT	

Table 2.3: **User_Roles**

Purpose: Maps users to roles (Many-to-Many)

Attribute	Data Type	Constraints
user_id	INT	FK
role_id	INT	FK

Table 2.4: **Cases**

Purpose: Stores investigation case details

Attribute	Data Type	Constraints
case_id	INT	PK
case_title	VARCHAR(255)	NOT NULL
case_description	TEXT	
owner_id	INT	FK

Attribute	Data Type	Constraints
status	VARCHAR(50)	
created_at	TIMESTAMP	

Table 2.5: **Evidence**

Purpose: Stores digital evidence details

Attribute	Data Type	Constraints
evidence_id	INT	PK
case_id	INT	FK
evidence_name	VARCHAR(255)	
evidence_type	VARCHAR(50)	
file_path	TEXT	
file_hash	VARCHAR(256)	
uploaded_by	INT	FK
uploaded_at	TIMESTAMP	

Table 2.6: **Chain_of_Custody**

Purpose: Tracks evidence handling

Attribute	Data Type	Constraints
custody_id	INT	PK
evidence_id	INT	FK
action_type	VARCHAR(100)	
performed_by	INT	FK
action_details	TEXT	
action_timestamp	TIMESTAMP	

Table 2.7: **Case_Members**

Purpose: Assigns users to cases

Attribute	Data Type	Constraints
case_id	INT	FK
user_id	INT	FK
assigned_at	TIMESTAMP	

Table 2.8: **Audit_Logs**

Purpose: Tracks system activity

Attribute	Data Type	Constraints
log_id	INT	PK
user_id	INT	FK

Attribute	Data Type	Constraints
action	VARCHAR(255)	
target_table	VARCHAR(100)	
target_id	INT	
action_details	TEXT	
action_timestamp	TIMESTAMP	

2.3 Creation of Database and Tables – DDL Commands

1. Creation of Database

```
CREATE DATABASE digital_evidence_db;
USE digital_evidence_db;
```

2. Creation of Users Table

```
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(50),
    department VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3. Creation of Roles Table

```
CREATE TABLE roles (
    role_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    role_name VARCHAR(50) NOT NULL,
    description TEXT
);
```

4. Creation of User_Roles Table

```
CREATE TABLE user_roles (
    user_id INT,
    role_id INT,
    PRIMARY KEY (user_id, role_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (role_id) REFERENCES roles(role_id)
);
```

5. Creation of Cases Table

```
CREATE TABLE cases (
    case_id INT AUTO_INCREMENT PRIMARY KEY,
    case_title VARCHAR(255) NOT NULL,
    case_description TEXT,
    owner_id INT,
    status VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(user_id)
);
```

6. Creation of Evidence Table

```
CREATE TABLE evidence (
    evidence_id INT AUTO_INCREMENT PRIMARY KEY,
    case_id INT,
    evidence_name VARCHAR(255),
    evidence_type VARCHAR(50),
    file_path TEXT,
    file_hash VARCHAR(256),
    uploaded_by INT,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (case_id) REFERENCES cases(case_id),
    FOREIGN KEY (uploaded_by) REFERENCES users(user_id)
);
```

7. Creation of Chain_of_Custody Table

```
CREATE TABLE chain_of_custody (
    custody_id INT AUTO_INCREMENT PRIMARY KEY,
    evidence_id INT,
    action_type VARCHAR(100),
    performed_by INT,
    action_details TEXT,
    action_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (evidence_id) REFERENCES evidence(evidence_id),
    FOREIGN KEY (performed_by) REFERENCES users(user_id)
);
```

8. Creation of Case_Members Table

```
CREATE TABLE case_members (
    case_id INT,
    user_id INT,
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (case_id, user_id),
    FOREIGN KEY (case_id) REFERENCES cases(case_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

9. Creation of Audit_Logs Table

```
CREATE TABLE audit_logs (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    action VARCHAR(255),
    target_table VARCHAR(100),
    target_id INT,
    action_details TEXT,
    action_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

2.4 Insertion of Tuples into the Tables – DML Commands

1. Insert into Users Table

```
INSERT INTO users (full_name, email, password_hash, role, department)
VALUES
('Rahul Sharma', 'rahul@gmail.com', 'hash123', 'Investigator', 'Cyber Crime'),
('Anita Singh', 'anita@gmail.com', 'hash456', 'Admin', 'Forensics'),
('Vikram Patel', 'vikram@gmail.com', 'hash789', 'Analyst', 'IT');
```

Users Table Output

user_id	full_name	email	role	department
1	Rahul Sharma	rahul@gmail.com	Investigator	Cyber Crime
2	Anita Singh	anita@gmail.com	Admin	Forensics
3	Vikram Patel	vikram@gmail.com	Analyst	IT

2. Insert into Roles Table

```
INSERT INTO roles (role_name, description)
VALUES
('Admin', 'Full system access'),
('Investigator', 'Handles investigation cases'),
('Analyst', 'Analyzes evidence');
```

Roles Table Output

role_id	role_name	description
1	Admin	Full system access
2	Investigator	Handles investigation cases
3	Analyst	Analyzes evidence

3. Insert into User_Roles Table

```
INSERT INTO user_roles (user_id, role_id)
VALUES
(1,2),
(2,1),
(3,3);
```

4. Insert into Cases Table

```
INSERT INTO cases (case_id, case_title, case_description, created_by, case_status,
created_at)
VALUES
('Cyber Fraud', 'Online scam investigation', 1, 'Open'),
('Data Breach', 'Unauthorized data access', 2, 'In Progress');
```

Cases Table Output

role_id	role_name	description	role_id
1	Admin	Full system access	1
2	Investigator	Handles investigation cases	2

5. Insert into Evidence Table

```
INSERT INTO evidence (case_id, evidence_name, evidence_type, file_path, file_hash,
uploaded_by)
VALUES
(1, 'Email Screenshot', 'Image', '/files/email.png', 'hash123abc', 1),
(2, 'Log File', 'Document', '/files/log.txt', 'hash456def', 2);
```

Evidence Table Output

evidence_id	case_id	evidence_name	evidence_type
1	1	Email Screenshot	Image
2	2	Log File	Document

6. Insert into Chain_of_Custody Table

```
INSERT INTO chain_of_custody (evidence_id, action_type, performed_by,
action_details)
VALUES
(1, 'Collected', 1, 'Collected from suspect device'),
(1, 'Analyzed', 3, 'Analyzed by forensic team');
```

7. Insert into Case_Members Table

```
INSERT INTO case_members (case_id, user_id) VALUES
(1,1), (1,3), (2,2);
```

8. Insert into Audit_Logs Table

```
INSERT INTO audit_logs (user_id, action, target_table, target_id, action_details)
VALUES
(1, 'INSERT', 'cases', 1, 'Created new case'),
(2, 'UPDATE', 'evidence', 2, 'Updated evidence details');
```

Audit Logs Table Output

log_id	user_id	action	target_table	target_id
1	1	INSERT	cases	1
2	2	UPDATE	evidence	2

CHAPTER 3

Complex Queries Based on Constraints, Sets, Joins, Views, Triggers and Cursors

3.1 Queries Based on Constraints

Question 1: UNIQUE Constraint

SQL STATEMENT:

```
ALTER TABLE users
ADD CONSTRAINT uq_email UNIQUE (email);

SELECT user_id, full_name, email FROM users;
```

Output

user_id	full_name	email
1	Rahul Sharma	rahul@gmail.com
2	Anita Singh	anita@gmail.com
3	Vikram Patel	vikram@gmail.com

Question 2: CHECK Constraint

SQL STATEMENT:

```
ALTER TABLE cases
ADD CONSTRAINT chk_status
CHECK (status IN ('Open','In Progress','Closed'));

SELECT case_id, case_title, status FROM cases;
```

Output

case_id	case_title	status
1	Cyber Fraud	Open
2	Data Breach	In Progress

Question 3: FOREIGN KEY Constraint

SQL STATEMENT:

```
ALTER TABLE evidence
ADD CONSTRAINT fk_case
FOREIGN KEY (case_id) REFERENCES cases(case_id);

SELECT evidence_id, case_id FROM evidence;
```

Output

evidence_id	case_id
1	1
2	2

3.2 Queries Based on Aggregate Functions

Question 1: Count total users

SQL STATEMENT:

```
SELECT COUNT(*) AS total_users FROM users;
```

Output

total_users
3

Question 2: Count evidence per case

SQL STATEMENT:

```
SELECT case_id, COUNT(*) AS total_evidence
FROM evidence
GROUP BY case_id;
```

Output

case_id	total_evidence
1	1
2	1

Question 3: Find max and min evidence ID

SQL STATEMENT:

```
SELECT MAX(evidence_id) AS max_id,
                  MIN(evidence_id) AS min_id
FROM evidence;
```

Output

max_id	min_id
2	1

3.3 Queries Based on Set Operations

Question 1: UNION

SQL STATEMENT:

```
SELECT user_id FROM users
UNION
SELECT user_id FROM case_members;
```

Output

user_id
1
2
3

Question 2: UNION ALL

SQL STATEMENT:

```
SELECT user_id FROM users
UNION ALL
SELECT user_id FROM case_members;
```

Output

user_id
1
2
3
1
3
2

Question 3: Intersection

SQL STATEMENT:

```
SELECT u.user_id
FROM users u
```

```
INNER JOIN case_members c
ON u.user_id = c.user_id;
```

Output

user_id
1
3
2

3.4 Queries Based on Subqueries

Question 1: **Users who uploaded evidence**

SQL STATEMENT:

```
SELECT full_name
FROM users
WHERE user_id IN (
SELECT uploaded_by FROM evidence
);
```

Output

full_name
Rahul Sharma
Anita Singh

Question 2: **Users managing cases**

SQL STATEMENT:

```
SELECT full_name
FROM users
WHERE user_id IN (
    SELECT created_by FROM cases
```

```
);
```

Output

full_name
Rahul Sharma
Anita Singh

Question 3: **Evidence uploaded by Rahul**

SQL STATEMENT:

```
SELECT evidence_name
FROM evidence
WHERE uploaded_by = (
    SELECT user_id FROM users
    WHERE full_name = 'Rahul Sharma'
);
```

Output

evidence_name
Email Screenshot

3.5 Queries Based on Joins

Question 1: **Users with roles**

SQL STATEMENT:

```
SELECT u.full_name, r.role_name
FROM users u
JOIN user_roles ur ON u.user_id = ur.user_id
JOIN roles r ON ur.role_id = r.role_id;
```

Output

full_name	role_name
Rahul Sharma	Investigator
Anita Singh	Admin
Vikram Patel	Analyst

Question 2: Cases with owner name

SQL STATEMENT:

```
SELECT c.case_title, u.full_name
FROM cases c
JOIN users u ON c.created_by = u.user_id;
```

Output

case_title	full_name
Cyber Fraud	Rahul Sharma
Data Breach	Anita Singh

Question 3: Evidence with case title

SQL STATEMENT:

```
SELECT e.evidence_name, c.case_title
FROM evidence e
JOIN cases c ON e.case_id = c.case_id;
```

Output

evidence_name	case_title
Email Screenshot	Cyber Fraud

evidence_name	case_title
Log File	Data Breach

3.6 Queries Based on Views

Question 1: Create user-role view

SQL STATEMENT:

```
CREATE VIEW vw_user_roles AS
SELECT u.full_name, r.role_name
FROM users u
JOIN user_roles ur ON u.user_id = ur.user_id
JOIN roles r ON ur.role_id = r.role_id;

SELECT * FROM vw_user_roles;
```

Output

full_name	role_name
Rahul Sharma	Investigator
Anita Singh	Admin
Vikram Patel	Analyst

Question 2: Create case view

SQL STATEMENT:

```
CREATE VIEW vw_case_details AS
SELECT c.case_title, u.full_name
FROM cases c
JOIN users u ON c.created_by = u.user_id;
SELECT * FROM vw_case_details;
```

Output

case_title	full_name	status
Cyber Fraud	Rahul Sharma	Open
Data Breach	Anita Singh	In Progress

Question 3 : Create evidence view

SQL STATEMENT:

```
CREATE VIEW vw_evidence AS
SELECT evidence_name, evidence_type
FROM evidence;

SELECT * FROM vw_evidence;
```

Output

evidence_name	evidence_type
Email Screenshot	Image
Log File	Document

3.7 Queries Based on Triggers

Question 1 : Default status trigger

SQL STATEMENT:

```
INSERT INTO cases (case_title, owner_id)
VALUES ('New Case', 1);
```

Output:

case_title	status
New Case	Open

Question 2 : Invalid evidence type

SQL STATEMENT:

```
INSERT INTO evidence
(case_id, evidence_name, evidence_type, file_path, file_hash, uploaded_by)
VALUES
(1, 'Fake File', 'invalid_type', '/evidence/fake.xyz', 'hash999', 1);
```

Output:

Error: Invalid evidence type

Question 3: Auto timestamp

SQL STATEMENT:

```
INSERT INTO audit_logs (user_id, action)
VALUES (1, 'LOGIN');
```

Output:

action	action_timestamp
LOGIN	(current time)

3.8 Queries Based on Cursors

Question 1: Display user names

SQL STATEMENT:

```
CALL show_users();
```

Output:

```
mysql> CALL show_users();
+---------+-------------+-------------------+---------------+-------+-------------+---------------------+
| user_id | full_name   | email             | password_hash | role  | department  | created_at          |
+---------+-------------+-------------------+---------------+-------+-------------+---------------------+
|       1 | Rahul Sharma | rahul@police.gov | h1            | NULL  | Cyber       | 2026-03-20 06:17:28 |
|       2 | Priya Singh | priya@lab.gov     | h2            | NULL  | Forensic    | 2026-03-20 06:17:28 |
|       3 | Arjun Mehta | arjun@police.gov  | h3            | NULL  | Cyber       | 2026-03-20 06:17:28 |
|       4 | Neha Kapoor | neha@court.gov    | h4            | NULL  | Judiciary   | 2026-03-20 06:17:28 |
+---------+-------------+-------------------+---------------+-------+-------------+---------------------+
4 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

Question 2: Display case titles

SQL STATEMENT:

```
CALL show_cases();
```

Output:

```
mysql> call show_cases();
+---------+-------------+--------------------+-----------+-------------+---------------------+
| case_id | case_title  | case_description   | created_by | case_status | created_at          |
+---------+-------------+--------------------+-----------+-------------+---------------------+
|       1 | ATM Fraud   | ATM skimming case  |         1 | Open        | 2026-03-20 06:17:41 |
|       2 | Data Breach | Company hack case  |         3 | Open        | 2026-03-20 06:17:41 |
|       3 | Mobile Theft | Phone theft case  |         1 | Closed      | 2026-03-20 06:17:41 |
+---------+-------------+--------------------+-----------+-------------+---------------------+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

Question 3: Display evidence names

SQL STATEMENT:

```
CALL show_evidence();
```

Output:

```
mysql> call show_evidence();
+-------------+---------+-----------------+---------------+-----------+-----------+-----------+---------------------+
| evidence_id | case_id | evidence_name   | evidence_type | file_path | file_hash | uploaded_by | uploaded_at         |
+-------------+---------+-----------------+---------------+-----------+-----------+-----------+---------------------+
|           1 |       1 | CCTV Footage    | video         | /e1.mp4   | hA        |         1 | 2026-03-20 06:18:00 |
|           2 |       1 | Transaction Logs | doc           | /e2.pdf   | hB        |         3 | 2026-03-20 06:18:00 |
|           3 |       2 | Server Logs     | doc           | /e3.txt   | hC        |         3 | 2026-03-20 06:18:00 |
|           4 |       2 | Malware File    | bin           | /e4.bin   | hD        |         2 | 2026-03-20 06:18:00 |
|           5 |       3 | Mobile Image    | image         | /e5.jpg   | hE        |         1 | 2026-03-20 06:18:00 |
|           6 |       1 | Audio File      | Audio         | NULL      | NULL      |         1 | 2026-03-20 06:32:27 |
+-------------+---------+-----------------+---------------+-----------+-----------+-----------+---------------------+
6 rows in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

CHAPTER 4

ANALYZING THE PITFALLS, IDENTIFYING THE DEPENDENCIES, AND APPLYING NORMALIZATIONS

(show the table before and after respective normalization)

4.1 Analyse the Pitfalls in Relations
4.2 First Normal Form
4.2.1: Identify Dependency
4.2.2: Apply Normalization to 1NF

4.3 Second Normal Form
4.3.1: Identify Dependency
4.3.2: Apply Normalization to 2NF

4.4 Third Normal Form
4.4.1: Identify Dependency
4.4.2: Apply Normalization to 3NF

4.5 BCNF
4.5.1: Identify Dependency
4.5.2: Apply Normalization to BCNF

4.6 Fourth Normal Form
4.6.1: Identify Dependency
4.6.2: Apply Normalization to 4NF

4.7 Fifth Normal Form
4.7.1: Identify Dependency
4.7.2: Apply Normalization to 5NF

CHAPTER 5

IMPLEMENTATION OF CONCURRENCY CONTROL AND RECOVERY MECHANISMS

5.1 Introduction to Transactions

5.1.1 Properties

5.1.2 States

5.2 Transaction Control Language (TCL)

5.2.1 Save point

5.2.2 Commit

5.2.3 Rollback

5.3 create 5 transactions for your project and execute

5.3.1 Transaction1 code with output (min 5 transactions)

Use savepoints, commit, rollback

< Sample:

```
-- Step 1: Start transaction by updating book availability
UPDATE Books
SET available_copies = available_copies – 1 WHERE book_id = 101;

-- Step 2: Set a savepoint after first change
SAVEPOINT after_first_update;

-- Step 3: Another update
UPDATE Books SET available_copies = available_copies – 1 WHERE book_id = 102;

-- Step 4: Oops! Something's wrong with second update, rollback to savepoint
ROLLBACK TO after_first_update;

-- Step 5: Continue with a safe update
UPDATE Books SET available_copies = available_copies + 1 WHERE book_id = 101;

-- Step 6: Commit all changes since savepoint
COMMIT; >
```

Sample 2:

UPDATE CLASS SET NAME= 'Tyler' where id= 101

SAVEPOINT A;

INSERT INTO CLASS VALUES (102, 'Zack');

Savepoint B;

INSERT INTO CLASS VALUES (103, 'Bruno')

Savepoint C;

Select * from Class;

The result will look like

Id	Name
98	Anita
99	Maria
100	Katilyn
101	Tyler
102	Zack
103	Bruno

Rollback to B;

SELECT * from Class;

Id	Name
98	Anita
99	Maria
100	Katilyn
101	Tyler
102	Zack

Now rollback to savepoint A

rollback to A;

SELECT * from class;

Id	Name
98	Anita
99	Maria
100	Katilyn
101	Tyler

5.3 Concurrency control

5.3.1 Concurrency control Algorithms

5.3.1 Locking commands

a. Row-Level Locking – SELECT ... FOR UPDATE

b. Table-Level Locking – LOCK TABLE

Lock Modes

Lock Mode	Description
`ROW SHARE`	Allows concurrent access; prevents other sessions from locking the table exclusively.
`ROW EXCLUSIVE`	Prevents other sessions from locking in share mode. Used by default for DML.
`SHARE`	Allows queries but not updates or deletes.
`SHARE ROW EXCLUSIVE`	A mix; more restrictive than `SHARE`.
`EXCLUSIVE`	Prevents all other access — full table lock.

c. COMMIT – Release All Locks

d. ROLLBACK – Undo Changes & Release Locks

5.3.2 Example (1 example for your project)

CHAPTER 6

FRONT-END AND BACK-END CODE OF <PROJECT TITLE>

6.1 Front–End Module codes

6.2 Database connectivity

CHAPTER 7

RESULTS AND DISCUSSIONS

7.1 Screenshots of front-end

7.2 Screenshots of Database

References